

HPC Lab4 Report

1 实验目的	3
2 评测约束	3
3 AMSS-NCKU 背景与 Baseline	3
3.1 程序流程	3
3.2 Baseline 与初始诊断	4
3.2.1 CPU perf 结果	4
3.2.2 GPU Nsight Systems 结果	4
4 TwoPuncture 初值求解优化	5
4.1 热点驱动和优化路线	5
4.2 失败尝试及其意义	6
4.3 GPU 路径的正确性修复	6
5 任务一：ABE CPU 演化优化	6
5.1 迭代一：从 43 s/步到固定成本分解	6
5.1.1 现象与假设	6
5.1.2 优化过程	7
5.1.3 验证与分析	7
5.2 迭代二：compute_rhs_bssn_ 算法重写	7
5.2.1 现象与假设	7
5.2.2 优化过程	8
5.2.3 验证与分析	8
5.3 失败尝试	9
6 任务二：ABEGPU GPU 演化优化	9
6.1 优化过程与热点定位	9
6.2 迭代一：先处理初值路径和 device call	9
6.2.1 现象与假设	9
6.2.2 优化过程	10
6.2.3 验证与分析	10
6.3 迭代二：拆分 RHS，并在寄存器与并行度之间找平衡	10
6.3.1 现象与假设	10
6.3.2 优化过程	10
6.3.3 验证与分析	11
6.4 迭代三：缩短 staged RHS 的 live range，并优化插值路径	11
6.4.1 现象与假设	11
6.4.2 优化过程	11
6.4.3 验证与分析	11
6.5 失败尝试与原因	12
6.6 最终结果与解读	13

7 思考题	13
7.1 思考题一：如何由 profiler 判断真正的优化对象？	13
7.2 思考题二：MPI 与 OpenMP 应如何组合？	13
7.3 思考题三：为什么提高 occupancy 可能使 GPU 更慢？	13
7.4 思考题四：Shared Memory 为什么不是 stencil 的必然答案？	13
7.5 思考题五：怎样区分合理浮点误差和程序错误？	13
7.6 思考题六：为什么短跑 A/B 可能给出错误结论？	14
7.7 思考题七：为什么端到端优化常常不是优化最显眼的 kernel？	14
7.8 思考题八：下一步最值得验证什么？	14

1 实验目的

本实验优化 AMSS-NCKU 数值相对论程序，模拟双黑洞并合的时空演化。程序从参数生成、TwoPuncture 初值求解、BSSN 时间演化到结果后处理构成一条完整的科学计算流水线。实验包含两个任务：任务一在华为鲲鹏 920B（ARM）上优化 TwoPunctureABE + ABE 的 CPU 端到端运行时间；任务二在单卡 NVIDIA A100 MIG 实例上优化 TwoPunctureABE + ABEGPU 的 GPU 端到端运行时间。两个任务共享 TwoPuncture 初值求解阶段。

2 评测约束

bssn_BH.dat 中两个黑洞的六个坐标列的相对 RMS 误差需小于 0.1%:

$$\text{RMS} = \sqrt{\frac{1}{|\Omega|} \sum_{i,j \in \Omega} \left(\frac{r_{i,j}^{\text{out}} - r_{i,j}^{\text{ref}}}{d_{i,j}} \right)^2} \leq 0.1\%, \quad (1)$$

其中 $d_{i,j} = \max(|r_{i,j}^{\text{ref}}|, |r_{i,j}^{\text{out}}|)$ ，忽略过小项后剩余比较项集合为 Ω 。同时，bssn_constraint.dat 中 Grid Level 0 的 Hamiltonian 与 momentum constraint 绝对值不超过 2.0:

$$\max_{t, c \in \{H, P_x, P_y, P_z\}} |C_c^{(0)}(t)| \leq 2.0. \quad (2)$$

3 AMSS-NCKU 背景与 Baseline

3.1 程序流程

AMSS-NCKU 的运行流程是一条流水线：AMSS_NCKU_Input.py 设置物理参数和计算参数，AMSS_NCKU_Program.py 读取输入并生成 parfile，TwoPunctureABE 求解双黑洞初始数据，随后根据 GPU_Calculation 选择 ABE（CPU）或 ABEGPU（GPU）进行 BSSN 时间演化，最后由 binary_output 整理结果。程序使用 BSSN 形式改写爱因斯坦场方程，通过四阶中心差分做空间离散、四阶 Runge-Kutta 做时间推进，并使用 AMR 在黑洞附近逐层加密网格。

CPU 与 GPU baseline 的统一对比如下。CPU 实测作业使用鲲鹏 920B、30 个 MPI rank、OMP_threads = 1，编译时采用 GNU 14.2、OpenMPI、-03，且未启用 OpenMP。GPU 实测作业使用 x86_64 主机上的 A100 1g.10gb MIG 实例、1 个 MPI rank、OMP_threads = 1，编译时显式启用 CUDA。

阶段	CPU baseline	GPU baseline
硬件与并行配置	Kunpeng 920B, 30 MPI rank, OpenMP 关闭	A100 1g.10gb MIG, 1 MPI rank, OpenMP 关闭
TwoPuncture 初值	4.83 min (290.1 s, 作业 67817)	4.93 min (295.6 s, 作业 68302)
BSSN 演化	约 28.7 min, 40 步估算	约 30.5 min, 100 步估算
结果整理	未完成	未完成
端到端总计	约 33.5 min	约 35.4 min
正确性检查	未执行, 作业超时	未执行, 作业超时

Table 1: CPU 与 GPU baseline 端到端时间对比

3.2 Baseline 与初始诊断

两个作业都受到 lab4 系列分区 30 min 最大墙钟时间的限制，因此均未生成完整输出。

CPU 作业 64220 完成了 35/40 个时间步，每步耗时 42.41–44.27 s，中位数 43.09 s。因此纯 CPU BSSN 演化时间估算为约 28.7 min ($43.09 \text{ s} \times 40$)，剩余 5 步约需 3.6 min。

TwoPuncture 初值求解实测 290.1 s (4.83 min，作业 67817)，编译约需 20 s，完整 CPU 端到端运行预计 33.5 min。

GPU 作业 65434 完成了 82/100 个时间步，82 步累计耗时约 1499.19 s，平均约 18.28 s，剩余 18 步估计还需约 5.5 min。因此纯 GPU BSSN 演化时间约为 30.5 min，加上 TwoPuncture 初值求解实测 295.6 s (4.93 min，作业 68302) 和编译、结果整理开销，完整 GPU 端到端运行预计为 35.4 min。

3.2.1 CPU perf 结果

为了在分区的 30 min 墙钟内完成 profiling，CPU 侧在独立 profiling 副本中将演化时间缩短为 5 步，配置仍保持 baseline 的 30 个 MPI rank、OpenMP 关闭和 -O3。

指标	CPU 5 步 perf stat
task-clock	6,806.64 s, 13.161 CPUs utilized
cycles / instructions	19.545e12 / 37.603e12, IPC = 1.92
branch-misses	58.852e9, 1.09%
L1-dcache-load-misses	71.838e9, 0.46%
LLC-load-misses	39.199e9, 41.58%
page-faults	4,761,160
elapsed time	517.19 s, 约 8.62 min

Table 2: CPU baseline 短 profiling 的 perf 计数器结果

这组计数器覆盖了 Python driver、TwoPuncture 和 5 步 ABE 演化，而不是只覆盖单个函数。IPC 为 1.92，分支预测失效率仅 1.09%，L1 数据缓存未命中率为 0.46%，但 LLC 未命中率达到 41.58%，说明访存层次和工作集局部性值得优先检查。

3.2.2 GPU Nsight Systems 结果

GPU 侧在 A100 1g.10gb MIG 实例上使用 5 步 profiling 副本执行。作业 67244 生成了下表的 Nsight Systems 统计：

对象	占比	累计时间或实例数
cudaDeviceSynchronize	CUDA API 96.5%	86.442 s, 5,116 次调用
cudaLaunchKernel	CUDA API 1.2%	1.082 s, 251,974 次调用
cudaMemcpy	CUDA API 1.2%	1.033 s, 6,432 次调用
rhs_kernel	GPU kernel 77.2%	68.674 s, 2,024 次实例
prolong3_kernel	GPU kernel 12.1%	10.774 s, 59,043 次实例
restrict3_kernel	GPU kernel 4.0%	3.552 s, 12,567 次实例
global_interp_kernel	GPU kernel 2.7%	2.390 s, 1,720 次实例
sommerfeld_rout_kernel	GPU kernel 2.0%	1.757 s, 46,560 次实例

Table 3: GPU baseline 短 profiling 的 Nsight Systems 结果

Nsight Systems 的内存传输统计显示，Host-to-Device 拷贝占传输时间 73.0%，Device-to-Host 拷贝占 22.4%，CUDA memset 占 4.6%。因此 GPU baseline 的首要诊断结论是：rhs_kernel 是主要计算热点，而大量同步和数据传输也显著影响端到端时间。

4 TwoPuncture 初值求解优化

4.1 热点驱动和优化路线

TwoPuncture 同时计入 CPU 与 GPU 两条评分路径。初期预分配、缩短预条件迭代和局部 GPU 化收益很小。进一步 profiling 显示，主要时间位于 relax 的矩阵装配与批量三对角求解，以及 chebft_Zeros 中重复计算的余弦变换。因此优化对象从容易并行的局部循环转向真实热点。

诊断	保留的实现	定性分析
LineRelax 反复索引并判断列归属	预打包 JFD 与列索引	减少间接访存和热循环分支，比增加线程更有效
chebft_Zeros 重复计算固定余弦表	缓存 Chebyshev 余弦表	用一次初始化消除迭代中的超越函数重复计算
relax 同奇偶类线相互独立	红黑 OpenMP 与 team-hoisting	沿独立线并行，同时保留 Thomas 递推顺序
GPU 构建强制 TwoP 走慢的部分 GPU 路径	解耦 TwoP 与 ABEGPU 的 GPU 宏	异构优化必须比较完整路径，局部 GPU 化不保证端到端更快

Table 4: TwoPuncture 的热点与实现对应关系

最终栈包含 packed JFD/cols、余弦表缓存和 AMSS_ENABLE_TWOP_OMP_TUNE。早期约 290 s 的基线经数据布局和余弦表优化降到约 186 s，红黑并行进一步降到约 73 s；team-hoisting 后，Intel 16 线程代表性结果约 27.8 s。数据来自不同平台与迭代阶段，不能直接拼成单一加速比，但清楚展示了瓶颈从装配、超越函数转移到线程管理的过程。输出在去除时间戳后哈希一致，Newton/BiCGStab 轨迹、裸质量和 ADM 质量保持不变。

4.2 失败尝试及其意义

尝试	现象	原因与启示
反复分配改为预分配	近乎无收益	固定小对象已由 glibc tcache 高效复用，分配不是热点
NRELAX 减半	单次迭代变快，总时间不变	预条件减弱导致 BiCGStab 迭代增加，局部收益被收敛变慢抵消
OpenMP 并行 J_times_dv	无收益	函数占比低且逐点部分受带宽限制，优化了非主导阶段
Derivatives_AB3 并行	NaN 或异常退出	余弦表懒初始化与 scratch 生命周期不具备线程安全性
局部 GPU 化 TwoP	仅小幅改善，慢于最终 CPU OpenMP	后确诊主因为 GPU 分支跳过 Derivatives_AB3, J*dv 用零导数而发散，修复见下节

Table 5: TwoPuncture 失败尝试汇总

4.3 GPU 路径的正确性修复

实验后期补测 AMSS_ENABLE_TWOP_GPU=0N 的独立求解时发现，GPU 路径的问题不是慢，而是数学上发散：BiCGStab 残余从初期的 1e-1 量级一路爆到 1e+107，CPU 同工况则一步收敛到 3.1e-13。检查调用链后定位到 TwoPunctures::J_times_dv 的 GPU 分支：它在调用 gpu_J_times_dv 后直接返回，跳过了 CPU 路径会先执行的 Derivatives_AB3。GPU kernel 消费 dv.d1 至 dv.d33 的谱导数，而这些字段从未被写入，于是实际施加的 Jacobian 导数项全为零，Newton 与裸质量迭代空转。

修复只有一行：在 GPU 调用前补上 Derivatives_AB3(nvar, n1, n2, n3, dv)。改动位于 #ifdef USE_GPU 内，CPU 构建不受任何影响；用 g++ -E -P 对比确认，无 USE_GPU 时修复前后源码的预处理器输出逐字节一致。

配置（同节点 16 核）	独立求解	收敛情况
CPU 路径（TWOP_GPU=OFF）	34.2 s	Newton it=1, F =3.1e-13
GPU 路径（修复前）	大于 13 min, 人工取消	残余爆到 1e+107, 发散
GPU 路径（修复后）	34.4 s / 34.1 s	Newton it=1, F =2.4e-13

Table 6: TwoPuncture GPU 路径修复前后的独立求解对比

5 任务一：ABE CPU 演化优化

5.1 迭代一：从 43 s/步到固定成本分解

5.1.1 现象与假设

早期 baseline 每步约 43 s，其中分析阶段远大于主演化。逐 rank profiling 进一步发现，OJ 中少数 rank 承担 global_interp 后成为 straggler，其余 rank 在集合通信处等待，因此“通信慢”其实是“计算不均衡”。固定成本侧，零步测试把 TwoPuncture 与网格初始化拆开：TwoPuncture 求解约 3.8 s（TWOP_OMP_TUNE 后），网格初始化约 31 s，剩余才是 40 步演化。由

此假设：优化应优先消除分析相位 straggler 与固定初值成本，而不是继续微调已经向量化的 RHS。

5.1.2 优化过程

按热点归因分四路推进。初值侧用 packed collective、余弦表与 OpenMP tune 降低每次提交的固定成本；编译侧叠加 -fno-tree-loop-distribute-patterns 与 -ftree-loop-im（合计约 -3.7%）；分析侧实现 DIST_INTERP，把插值点跨 30 个 rank 分片后汇总；负载侧用 LOADBAL 按 patch 点数重新分配。

5.1.3 验证与分析

最终 OJ 覆盖 40 个轨迹时间组和 236 个有效项，轨迹 RMS 为 0；Hamiltonian 最大值为 0.2774，三个动量约束最大值均远低于 2，正确性全程 bit-exact。下表是最终优化栈与端到端演进。

层次	最终方案	为何有效
初值	TwoP packed、余弦表与 OpenMP tune	降低每次提交必须承担的固定成本
主演化	-fno-tree-loop-distribute-patterns + -ftree-loop-im	改善差分循环生成代码，避免不利的 loop-distribute 模式
分析	DIST_INTERP	将插值点跨 30 rank 分片并汇总，消除单 rank 串行热点
负载	LOADBAL	按 patch 点数分配，降低最慢 rank 的计算量
并行配置	30 MPI × 1 OMP	一 rank 对应一个物理核，避免 SMT 与线程访存争用

Table 7: ABE CPU 最终优化栈

DIST_INTERP 的收益最大，并非减少了总计算量，而是把少数 rank 的工作均匀分散。集合通信 wall time 随之下降，证明此前所谓的通信热点主要是等待慢 rank。LOADBAL 可继续叠加，但 AMR patch 是粗粒度任务，只能缓解而不能完全消除 straggler。

状态	端到端时间	OJ 分数	主要变化
早期正式提交	909.42 s	20	尚未形成完整优化栈
flag 与 TwoP 阶段	约 568 s	48	固定成本与 RHS 编译改善
DIST_INTERP + LOADBAL	408.915 s	86	消除分析 straggler 与负载重分配

Table 8: 任务一端到端结果演进

此时 408.9 s 距 340 s 档位还差约 68.9 s。零步测试把固定开销分解为约 3.8 s TwoPuncture 与约 31 s 网格初始化，40 步演化约 9.35 s/步，因此 340 s 要求每步 ≤ 7.6 s，即必须再砍约 1.7 s/步。瓶颈已明确收敛到主演化本身。

5.2 迭代二：compute_rhs_bssn_ 算法重写

5.2.1 现象与假设

演化相位 9.35 s/步里，compute_rhs_bssn_ 约占 30%。此前用 perf 看到 IPC 1.69、load:FP 3.4:1，据此判断它是数据依赖地板，随后 26 个编译杠杆（unroll、split、gcse、modulo-

sched、prefetch、LTO、PGO) 全部实测 0 收益，于是短期结论是“约束内不可达”。但这个判断混淆了两件事：编译器已最优只说明“对这批 whole-array 语句调度已到极限”，并不说明“这批语句的算法结构本身最优”。

再往下拆，compute_rhs_bssn_ 里约 80 个三维中间数组（度规、导数、Christoffel、Ricci 的中间量）以 whole-array 方式逐个物化，每次调用光数组就是约 6.6 MB。gfortran 虽然把每个赋值向量化，但整批中间量反复进出 L1/L2，per-call 计算被 cache-miss 拖到约 19 ms。假设：把 whole-array 改写为显式循环、并把 42 个导数数组从三维物化改成 k 方向滚动窗口，工作集从约 6.6 MB 压进 L2（约 1 MB），即可兑现“不保证可达”的收益。

5.2.2 优化过程

重写分两阶段，全程保持 bit-exact。第一阶段（点态融合）把 5 个点态 init (alpn1/chin1/gxx/gyy/gzz/div_beta) 改点态标量，24 个 RHS 方程改成显式 do k/j/i 嵌套循环。第二阶段（k-滚动导数融合）把 21 个 fderivs 调用内联为 fderivs_plane/fdderivs_plane 子程序加 frx 反射助手，42 个导数数组从三维物化改为 k 切片滚动窗口，不再全量落盘。

前置还修掉一个栈溢出：lev8 深调用栈下，80 个 runtime-sized 自动数组被 gfortran 强制放栈（-fno-automatic 无效），每调用约 6.6 MB，深递归直接 segfault。改为 allocatable（堆分配）并在所有退出路径 deallocate 后才可运行。

5.2.3 验证与分析

两阶段结果对比鲜明：点态融合 bit-exact (RMS=0) 但反而约 +20% 慢（约 11 s/步），因为只消除了约 20 个点态数组，42 个导数数组仍物化，显式循环的寄存器依赖净效果为负；k-滚动融合则真正把工作集压进 L2，5 步短跑 bit-exact，40 步全量 (job 151519) FINAL PASS、RMS=0，avg 7.27 s/步，Program Cost 297.31 s。部署到 ~/lab4-cpu 后复验 40 步 284.78。

关键点是显式循环与 whole-array 在 gfortran 下编译为相同求和序，因此重写全程没有引入任何浮点重排，RMS 始终为 0。

阶段	每步时间	相对 baseline	正确性
baseline (flag+loopim)	约 9.0 s	基准	RMS=0
Stage 1a 点态融合	约 11 s	+20% 更慢	RMS=0
Stage 1b k-滚动融合	约 6.9 s	-23%	RMS=0

Table 9: compute_rhs 重写两阶段对比（5 步短跑稳态）

OJ 最终 300.943 s、120 分满分，轨迹 RMS=0、约束最大值与重写前逐位一致。改进幅度为 -108 s (-26%)，主要来自主演化从约 9.35 s/步降到约 6.98 s/步。

项	重写前	重写后
OJ wall	408.915 s	300.943 s
OJ 分数	86/120	120/120
trajectoryRMS	0	0
Ham max	0.27739667	0.27739667
Px/Py/Pz max	0.0281/0.0315/0.0265	完全一致

Table 10: OJ 实测前后对比

5.3 失败尝试

类别	代表尝试	定性分析
增加核内并行	-march=native、OpenMP fderivs	大量三维数组已形成带宽压力，额外向量加载或线程只会加剧 cache 竞争
改变进程线程比	15×2、10×3、60×1	减少 rank 损失 patch 并行度，SMT 又共享执行与缓存资源
循环重写	Ricci 融合、graphite 分块	融合扩大活跃工作集并增加循环管理；而点态融合（Stage 1a）也证明只改循环不砍数组物化反而更慢
分布式 RHS	k-slice 30-rank / 2-rank 复制	跨切片中间量 stale-read 破坏正确性（RMS 0.16），且 Bcast 全量数组的带宽开销超过计算收益
通信微调	合并、异步、移除 barrier	Sync 多数是等待计算 straggler，减少消息不能缩短最慢 rank
细粒度负载均衡	切分 patch、owner/helper RHS	管理与边界成本上升，跨切片中间量还会产生 stale-read 并破坏正确性
激进编译	PGO、LTO、unroll、prefetch	多数中性，部分组合改变结果或使代码更慢，收益不能机械叠加

Table 11: 任务一失败路径及共同原因

这些失败把“通信慢”修正为“计算不均衡导致 barrier 等待”，把“IPC 1.69 是数据依赖地板”修正为“编译器调度已最优但算法结构可改”。分布式 RHS 在两种 rank 数下都因 k 切片边界的 stale-read 失败，说明跨 rank 复制这条路从根上不可行；真正的解法是在单 rank 内部把中间数组的物化方式改掉，这正是 k-滚动融合所做的。

6 任务二：ABEGPU GPU 演化优化

6.1 优化过程与热点定位

最终正式源使用 CPU TwoPuncture 初值路径，GPU 只负责 BSSN 演化；真实 OJ 配置为 1 个 MPI rank、8 个 OpenMP 线程、100 个时间步。

最初 rhs_kernel 占 GPU kernel 时间约 69%，prolong3_kernel 约 16.5%。rhs_kernel 的自然寄存器需求约 250 个/thread，__launch_bounds__(256,2) 将其压到 128 个寄存器，但 occupancy 只有约 21.5% 至 25%，L1TEX scoreboard stall 和 No Eligible warp 较高。这个现象说明瓶颈是数据依赖造成的等待，不是简单的显存带宽不足。

6.2 迭代一：先处理初值路径和 device call

6.2.1 现象与假设

早期 GPU 版本的总时间很长，但 GPU 演化本身并没有占完全部时间。进一步看调用链后发现，TwoPuncture 初值求解也被迫走了较慢的部分 GPU 路径，而 GPU 资源只有一张 A100 MIG 卡。于是我先假设：如果 TwoPuncture 改回经过验证的 CPU OpenMP 路径，就能把 GPU 留给真正适合并行的 BSSN 演化，而且不会改变物理计算。

同时，Nsight Compute 显示 rhs_kernel 中有跨翻译单元的 stencil helper 调用。这个大 kernel 的自然寄存器需求约为 250 个/thread，warp 经常在等依赖数据。我的第一个代码层面尝试是把四个热点 helper 放进 header 并强制内联，让编译器看到更完整的 load 和计算关系；随后再试安全的 branchless fh，减少边界分支，但保留合法地址计算。

6.2.2 优化过程

正式构建中设置 AMSS_ENABLE_TWOP_GPU=OFF，同时打开 AMSS_ENABLE_TWOP_OMP_TUNE、AMSS_ENABLE_PACKED_RELAX 和余弦表缓存，让 TwoPuncture 走 CPU 快速路径。GPU 侧把 helper 移入 header，使用 __forceinline__，再对 fh 的反射系数选择做 branchless 化。这里没有把所有判断都删除，越界风险较高的路径仍然保留 early return。

6.2.3 验证与分析

helper 内联把端到端时间从约 1266 s 降到 1052 s，branchless fh 又降到约 1007 s，两次都通过位级检查。这个结果说明原来的 device call 边界确实妨碍了编译器安排访存，但也说明“减少分支”本身不是充分条件，地址安全必须同时保留。CPU TwoPuncture 的解耦也成为后面所有 GPU 候选的共同基础，因为它减少了固定开销，且没有把 GPU 演化结果混入初值误差。

改动	实测结果	保留理由
TwoPuncture 改走 CPU OpenMP	初值阶段明显缩短	避免 GPU 资源被较慢的初值路径占用，物理检查保持通过
stencil helper __forceinline__	约 1266 s 降至 1052 s	消除跨翻译单元 device call，允许编译器重排访存和计算
安全 branchless fh	约 1052 s 降至 1007 s	减少边界分支，同时保留合法地址和 early return

Table 12: ABEGPU 迭代一的改动与验证结果

6.3 迭代二：拆分 RHS，并在寄存器与并行度之间找平衡

6.3.1 现象与假设

profiling 中 rhs_kernel 约占 GPU kernel 时间的 69%，prolong3_kernel 约占 16.5%。但 rhs_kernel 并不是显存带宽已经跑满，而是 128 个寄存器/thread、约 21.5% 至 25% occupancy，以及较高的 L1TEX scoreboard stall 共同造成延迟。最初我以为增加驻留 block 就能隐藏等待，于是尝试改变 launch_bounds；结果显示，寄存器压得越低，spill 越多，反而更慢。于是新的假设变成：应先把 interior 点和 boundary/face 点分开，减少无效的边界判断，再保留一个能控制 spill 的 block 配置。

6.3.2 优化过程

我把 RHS 分成 interior 和 boundary/face 路径，interior kernel 使用 __launch_bounds__(256,2)，face 路径单独处理对称边界。演化辅助阶段同时保留 P31 的 global interpolation 变量批处理、P32 的 Sommerfeld 紧凑发射、P33 的 2x2x2 prolong3 tap reuse，以及 A38-1 的 fused-z 插值。此阶段的重点是减少 kernel 内部的有效工作和临时数据。

6.3.3 验证与分析

Milestone B 的短 A/B 得到 $F=1.128$ 并保持 bit-exact, 后续 100 步检查也通过。P313233 组合的 Program Cost 约为 539.48 s, A38-1 fused-z 后约为 534.31 s; 同一正式栈的真实 OJ 结果为 536.760 s、85.20 分。这里能看出一个容易误判的地方: SASS 指令减少和短跑加速是有意义的, 但不等于正式 OJ 时间会按同样比例下降, 缓存、TwoPuncture 固定成本和节点差异都会影响总数。

阶段	结果	分析
interior/boundary split	短 A/B $F=1.128$, 100 步检查通过	减少 interior 点上的边界判断, 保留不同区域的正确地址逻辑
P313233 组合	Program Cost 约 539.48 s	global interpolation、Sommerfeld 和 prolong3 的批处理共同降低固定开销
A38-1 fused-z	约 534.31 s, 检查通过	删除局部 6^3 插值数组, 减少中间数据和访存压力
真实 OJ	536.760 s, 85.20 分	与缓存运行区分, 说明稳定候选仍受正式固定开销和节点状态影响

Table 13: ABEGPU 迭代二的区域拆分与辅助 kernel 优化

6.4 迭代三: 缩短 staged RHS 的 live range, 并优化插值路径

6.4.1 现象与假设

在前两轮之后, 单纯继续调 launch_bounds、shared memory 或 stream 已经没有稳定收益。新的 nsys 结果显示, RHS interior 的 advection 阶段和 global interpolation 仍然值得处理。我的判断是, 重复传递索引、stride、坐标和速度值会让中间量活得太久; 而 $ORDN == 6$ 的 global interpolation 每次都走通用 Neville 过程, 固定节点下存在可消除的通用工作。

6.4.2 优化过程

RHS interior 改用 staged path, 把几何计算、核心演化和 advection 分开; 在此基础上, direct-first 版本把一阶导数需要的参数外提, 并在 advection 中直接使用 stride/offset, 减少重复的索引元数据。global interpolation 则只在 $ORDN == 6$ 时使用固定节点的五次 Lagrange 评价, 其他阶数继续走原来的 d_gi_fused fallback。这样既利用了本题固定阶数的事实, 也没有把其他插值情况一起改掉。

6.4.3 验证与分析

advection 复用的同卡 A/B 中位数从 4.9409 s 降到 4.61515 s, 完整缓存验证约 457.055 s, check.sh FINAL PASS, 100/100 轨迹通过且 RMS 为 0。direct-first 组合的缓存全量结果为 338.639048 s, 仍通过完整检查。最后的 GI-Lagrange6 A/B 把 Step2 中位数从 3.33000 s 降到 2.95546 s, $F=1.126728$; 缓存全量验证为 305.069513 s, check.sh FINAL PASS。

改动	实测结果	正确性与决定
staged RHS + advection reuse	A/B 4.9409 s 降至 4.61515 s; 全量约 457.055 s	check.sh FINAL PASS, 100/100 轨迹通过, 保留
direct-first 组合	缓存全量 338.639048 s	check.sh FINAL PASS, 作为 GI-Lagrange6 的基础, 保留
GI-Lagrange6	Step2 3.33000 s 降至 2.95546 s, F=1.126728; 缓存全量 305.069513 s	100 步检查通过

Table 14: ABEGPU 迭代三的 staged RHS 与插值优化结果

6.5 失败尝试与原因

尝试	结果	原因与处理
第一次 512-thread block	短测快, 但 RMS=103.2%	寄存器资源不足, kernel 未正常工作, 直接淘汰
launch_bounds(256, 3/4)	比 (256, 2) 慢约 10% 至 15%	寄存器预算过低, spill 到 local memory, 额外 warp 无法抵消延迟
两路 RHS 拆分	约慢 3%, spill 更严重	增加 scratch 和 launch, 却没有真正缩短 live set, 关闭
__ldg、shared staging、load hoisting、软件流水	中性或变慢	RHS 是依赖等待而非简单带宽不足, 新增管理和访存成本抵消收益
per-stream sync、prolong3 Z 累加重写、face launch fusion	同卡 A/B 近中性	没有稳定端到端收益, 不保留
4x4x4 tap sharing	出现 race 和 divergent barrier	并行同步结构不安全, 停止该方向
无条件 branchless 阶数选择	step 28 越界	mask 不能保护非法 load, 恢复 early return 并钳制索引
混合 FP32	局部约快 3.7%, RMS 约 0.00715	超过 0.001 容差, 关闭
A74 face 轴映射、A75 缓存隔离	分别编译失败、quota 超限	没有运行性能结论, 记录为设置/资源失败

Table 15: ABEGPU 失败尝试及其处理结果

这些尝试说明, GPU 优化不能只看某个短测数字。512-thread 的第一次尝试看起来很快, 实际因寄存器资源不足导致 kernel 没有正常工作; 把 launch_bounds 压到 3 或 4 个 block 又引入了更多 spill。两路 RHS 拆分、shared memory staging、__ldg、load hoisting 和软件流水也没有得到稳定收益, 原因是新增的 scratch、launch 或 local-memory 访问抵消了减少依赖的好处。

其他方向同样被实测关闭: per-stream synchronization、prolong3 的 Z 累加重写、face launch fusion、不同 block shape 和过度的 tile sharing 都在同卡 A/B 中接近中性或变慢; 4x4x4 tap sharing 还出现 shared-memory race 和 divergent barrier。无条件 branchless 阶数

选择在短跑中较快，却在 step 28 触发越界，修复后必须保留 early return 并钳制地址。混合 FP32 只得到约 3.7% 的局部加速，trajectory RMS 约 0.00715，超过 0.001 容差，因此不能保留。A74 的 face 轴映射在编译阶段失败，A75 则因远端 home quota 超限未进入构建，这两项没有运行性能结论。

6.6 最终结果与解读

最终实际 OJ 记录为 351.956766 s、102.032239 分，展示为 102/120。100/100 个轨迹时间点和约束检查均完成，trajectoryRMS=0，level-0 的 Hamiltonian、三个 momentum constraint 最大值分别为 0.28974817、0.039343259、0.047298107，全部通过。

从早期 1228.53 s 到最终 351.96 s，主要收益来自 CPU/GPU 初值路径解耦、RHS 的内外区域拆分、减少 device call 和 live metadata，以及把固定阶数插值改成更直接的评价。

7 思考题

7.1 思考题一：如何由 profiler 判断真正的优化对象？

我一开始也差点把时间最多的同步调用当成了热点。后来对照逐 rank 和 kernel 的数据才发现，CPU 的 MPI_Allreduce 主要是在等 global_interp 较慢的 rank，GPU 的 cudaDeviceSynchronize 主要是在等 kernel 做完。所以我现在会先思考“谁在等谁”，再决定改通信、负载还是 kernel。

7.2 思考题二：MPI 与 OpenMP 应如何组合？

这次实验给我的感觉是，MPI 和 OpenMP 没有一个可以直接套用的比例。ABE 用 30 个 MPI rank、每个 rank 1 个线程比较合适，因为 patch 并行度还在，也没有 SMT 争用；TwoPuncture 的红黑线没有 MPI 通信，反而适合在一个进程里用 OpenMP。我选择并发多枚举尝试。

7.3 思考题三：为什么提高 occupancy 可能使 GPU 更慢？

原因是 rhs_kernel 的寄存器预算被压低后，变量被 spill 到 local memory，新增的读写延迟比多驻留几个 warp 带来的好处更大。

7.4 思考题四：Shared Memory 为什么不是 stencil 的必然答案？

我原来觉得 stencil 很适合 shared memory，但这次试下来不能这么简单判断。ABEGPU 的 RHS 同时读很多场，做 tile 还要装 halo、同步和计算地址，缓存一部分 Christoffel 中间量省下的 global load 很快就被这些开销抵消了。以后如果再试，我会先量重复访问最多的字段，再决定是否值得做 tile，而不是整块数据一起搬进去。

7.5 思考题五：怎样区分合理浮点误差和程序错误？

我会先看完整轨迹 RMS 是否随时间变大，再看各 refinement level 的 Hamiltonian 和 momentum constraint，最后查 NaN、launch error、RHS 是否全为零以及轨迹有没有冻结。比如 512-thread 那次不是误差有点大，而是 kernel 根本没有正常工作；FP32 的 RMS 也已经超过容差；branchless 版本则是后期越界。

7.6 思考题六：为什么短跑 A/B 可能给出错误结论？

短跑只能帮我筛选方向，不能直接宣布成功。AMR 网格会动，前几步的负载和后期不一样，节点波动也可能造成几个百分点的假收益。per-stream sync 看起来有收益但重复 A/B 后消失，branchless 版本则跑到 step 28 才越界，所以我会先做同一作业内交错 A/B，再用 100 步和 check.sh 收尾。

7.7 思考题七：为什么端到端优化常常不是优化最显眼的 kernel？

这次最明显的例子是 TwoPuncture。它不是 GPU 演化的最大 kernel，却通过改成 CPU 的快速初值路径省了很多端到端时间；CPU 侧也是先处理分析阶段的 straggler，最后才做 compute_rhs_bssn_ 的大改。我的理解是，评分看的是整条流水线，所以优化优先级应该按改动谁的效果最好来排，而不是按哪个函数最容易改来排。

7.8 思考题八：下一步最值得验证什么？

如果继续做，我会先检查 GPU 的 face RHS 和剩余的 global interpolation，因为它们在新剖析里还占比较高。CPU 这边已经靠 k-滚动导数融合把主演化降到约 6.98 s/步并拿到 120 分，继续堆编译 flag 的意义不大。GPU 的下一步我会先把 SASS 和源码对应起来，确认到底是 live set、依赖 load 还是小 kernel 调度在拖慢，再一次只改一个变量，最后用端到端时间和完整 RMS 决定留不留。